

Chapter 6

Specialized Instrument and Devices

Contents

Camera

- Use an existing camera app

- Build your own camera function

Media

Sensors

Camera

The integration of cameras into mobile devices has revolutionized the way we interact with technology.

Basic app features:

1. Visual content creation - take photos and videos
2. Document scanning
3. QR code scanning

Camera

Advanced features:

1. Augmented Reality (AR): Cameras can be used to overlay digital content onto the real world, creating immersive experiences.
2. Image Recognition: Apps can recognize objects, text, or faces in images to provide additional information or functionality.
3. Facial Recognition: This technology can be used for secure authentication, unlocking devices, or enabling personalized experiences.

What can your phone camera do?

Claim vehicle insurance

Cheque deposit

Search

மொழிபெயர்

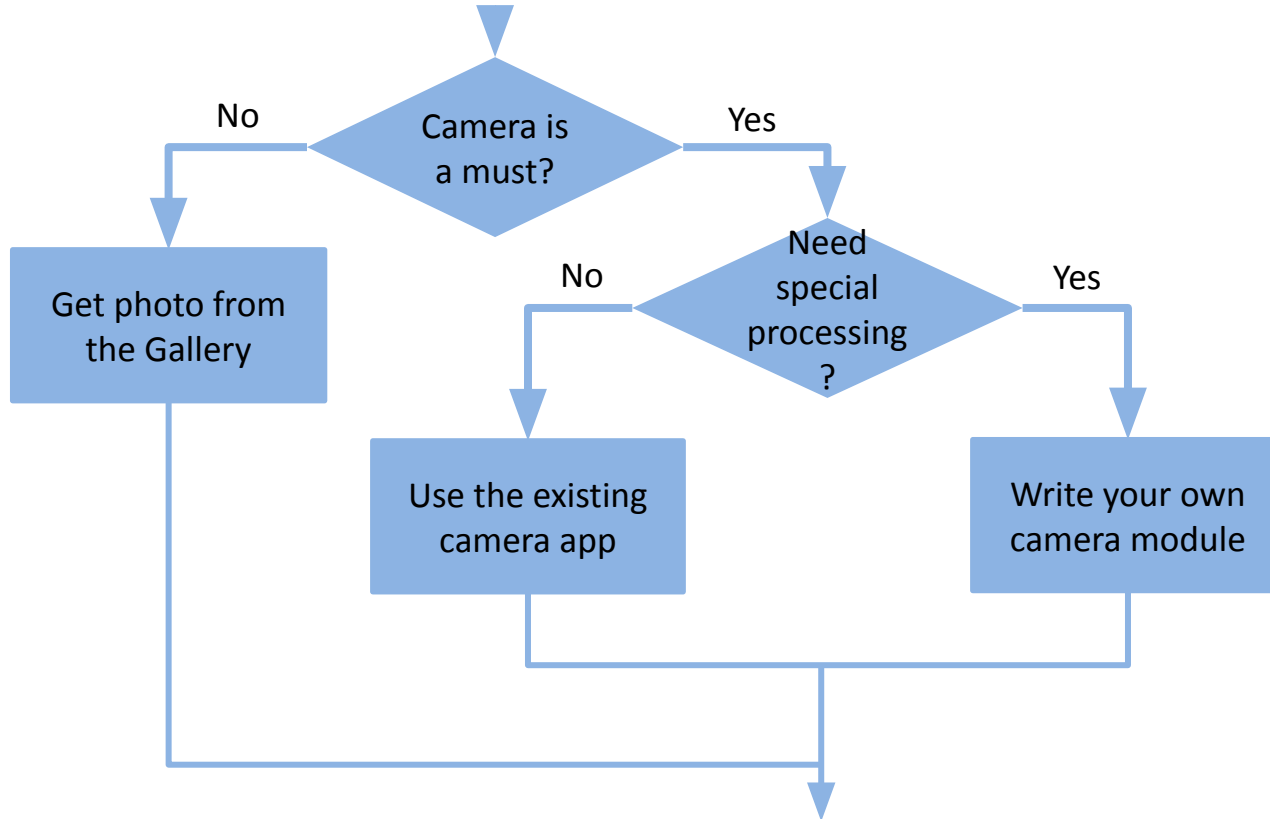
Eye test



5

<https://developers.google.com/ml-kit/vision/text-recognition>

Camera - Considerations



Camera Methods

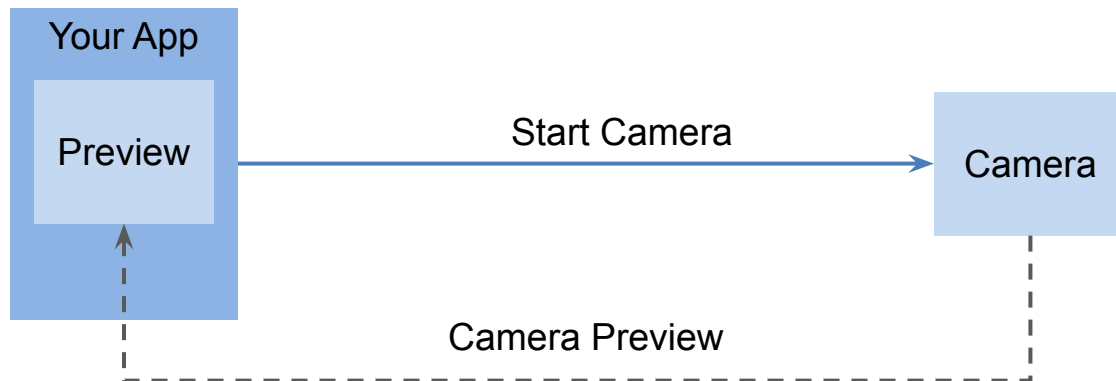
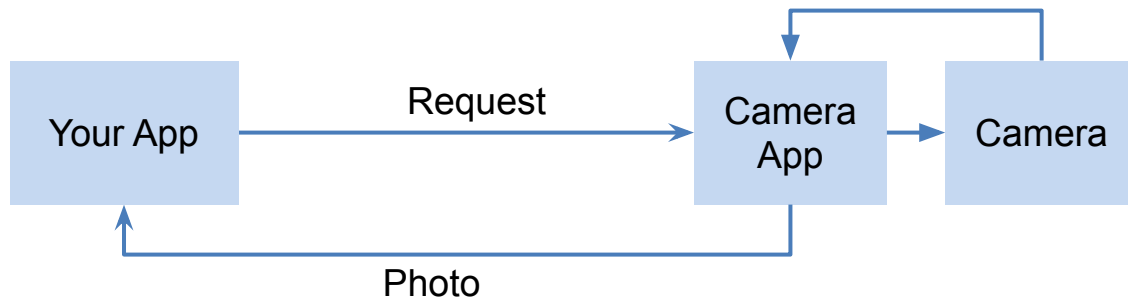
Use an existing camera app

- Simple implementation - using existing packages
- Obtain a photo from the camera app or the gallery
- No need permission

Build your own camera function

- Complex implementation - write your own code
- Create a camera preview
- Request the Camera permission

Camera Methods



Use an existing camera app

Use an existing camera app

Flutter uses the `image_picker` to capture images and videos from the device's camera or gallery.

Insert dependencies:

```
image_picker: ^version_code
```

Import package:

```
import 'package:image_picker/image_picker.dart';
```

Use an existing camera app

You may also include the following to find path to store images or to access the device file path:

`path_provider`

Finds the correct paths to store images.

`path`

Creates paths that work on any platform.

Use an existing camera app

```
Future<void> _pickImage() async {  
  final pickedFile = await ImagePicker().pickImage(source: ImageSource.camera);  
  if (pickedFile != null) {  
    setState(() {  
      _image = File(pickedFile.path);  
    });  
  
    // Save the image to the device's gallery (Android-specific)  
    if (Platform.isAndroid) {  
      final galleryPath = '/sdcard/DCIM/Camera';  
      final destinationFile =  
        File('$galleryPath/${DateTime.now().millisecondsSinceEpoch}.jpg');  
      await pickedFile.saveTo(destinationFile.path);  
    }  
  }  
}
```

Build your own camera function

Build your own camera function

Camera Features

In the context of Android, to prevents your app from being installed to devices that do not include a camera

```
<uses-feature android:name="android.hardware.camera" />
```

Build your own camera function

Storage Permission

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion = "18" />
```

Audio Recording Permission

```
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

Location Permission

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

Build your own camera function

The camera plugin provides tools to get a list of the available cameras, display a preview coming from a specific camera, and take photos or videos.

Steps:

1. Add the required dependencies.
2. Get a list of the available cameras.
3. Create and initialize the `CameraController`.
4. Use a `CameraPreview` to display the camera's feed.
5. Take a picture with the `CameraController`.
6. Display the picture with an `Image` widget.

Build your own camera function

Step 1: Add the required dependencies.

camera

- Provides tools to work with the cameras on the device.

path_provider

- Finds the correct paths to store images.

path

- Creates paths that work on any platform.

Build your own camera function

Step 1: Add the required dependencies.

To add the packages as dependencies, run flutter pub add:

```
$ flutter pub add camera path_provider path
```

Build your own camera function

Step 1: Add the required dependencies.

Android

- Update minSdkVersion to 21 (or higher).

Build your own camera function

Step 1: Add the required dependencies.

iOS

- Inside ios/Runner/Info.plist in order to access the camera and microphone.

```
<key>NSCameraUsageDescription</key>
```

```
<string>Explanation on why the camera access is needed.</string>
```

```
<key>NSMicrophoneUsageDescription</key>
```

```
<string>Explanation on why the microphone access is needed.</string>
```

Build your own camera function

Step 2: Create and initialize the CameraController.

```
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized(); // Ensure that plugin services are initialized  
  
  // Obtain a list of the available cameras on the device.  
  final cameras = await availableCameras();  
  
  // Get a specific camera from the list of available cameras.  
  final firstCamera = cameras.first;  
  
  runApp(  
    MaterialApp(  
      theme: ThemeData.dark(),  
      home: TakePictureScreen(  
        // Pass the appropriate camera to the TakePictureScreen widget.  
        camera: firstCamera,  
      ),  
    ),  
  );  
}
```

Build your own camera function

Step 2: Create and initialize the CameraController.

```
// A screen that allows users to take a picture using a given camera.
class TakePictureScreen extends StatefulWidget {
  const TakePictureScreen({
    super.key,
    required this.camera,
  });

  final CameraDescription camera;

  @override
  TakePictureScreenState createState() => TakePictureScreenState();
}
```

Build your own camera function

Step 3: Create and initialize the CameraController.

```
class TakePictureScreenState extends State<TakePictureScreen> {  
    late CameraController _controller;  
    late Future<void> _initializeControllerFuture;  
  
    @override  
    void initState() {  
        super.initState();  
        _controller = CameraController( // Create a CameraController.  
            Widget.camera,              // Get a specific camera  
            ResolutionPreset.medium,    // Define the resolution to use.  
        );  
        _initializeControllerFuture = _controller.initialize();  
    }  
    ...  
}
```

Build your own camera function

Step 3: Create and initialize the CameraController.

```
class TakePictureScreenState extends State<TakePictureScreen> {  
    ...  
    @override  
    void dispose() {  
        _controller.dispose(); // Dispose of the controller  
        super.dispose();  
    }  
  
    @override  
    Widget build(BuildContext context) {  
        return Container(); // Fill this out in the next steps.  
    }  
}
```


Build your own camera function

Step 4: Use a CameraPreview to display the camera's feed

```
FutureBuilder<void>(
  future: _initializeControllerFuture,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.done) {
      // If the Future is complete, display the preview.
      return CameraPreview(_controller);
    } else {
      // Otherwise, display a loading indicator.
      return const Center(child: CircularProgressIndicator());
    }
  },
)
```

Build your own camera function

Step 5 : Take a picture with the CameraController

```
FloatingActionButton(  
  onPressed: () async {  
    try {  
      await _initializeControllerFuture; // Ensure that the camera is initialized.  
      final image = await _controller.takePicture(); // Attempt to take a picture  
    } catch (e) {  
      // If an error occurs, log the error to the console.  
      print(e);  
    }  
  },  
  child: const Icon(Icons.camera_alt),  
)
```

Build your own camera function

Step 6: Display the picture with an Image widget

```
Image.file(File(imagePath)),
```

Google ML Kit

Google's ML Kit is a mobile SDKs that allows developers to easily integrate machine learning capabilities into their Android, iOS, and web applications.



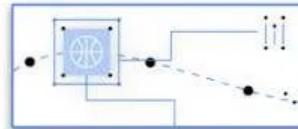
Text
recognition



Image
labeling



Barcode
scanning



Object
detection
and tracking



Face
detection

Media

Media

Mobile devices maintain separate audio streams for playing music, alarms, notifications, the incoming call ringer, system sounds, in-call volume, etc

Most of these streams are restricted to system events

We focus on playing audio using the music stream

Media

Plays local (assets) and external (streaming) files

Supports any media codec that is provided by the platform and those that are device-specific

Recommendation: use core media formats

Use the `video_player` plugin to play videos stored on the file system, as an asset, or from the internet (Note: Not available on Linux and Windows)

Media

Permission declaration

If you are using MediaPlayer to stream network-based content

```
<uses-permission android:name="android.permission.INTERNET" />
```

If your player application needs to keep the screen from dimming or the processor from sleeping

```
<uses-permission android:name="android.permission.WAKE_LOCK" />
```


Media

Core media file formats:

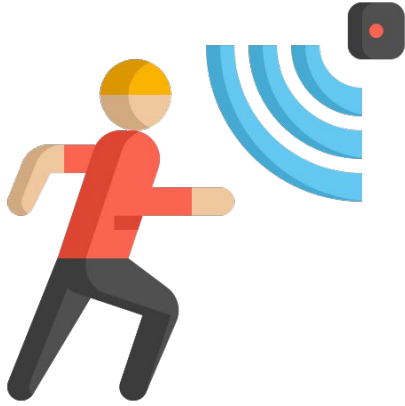
Audio: .3gp .mp3 .mp4 .mid .wav .ogg

Picture: .jpg .gif .png .bmp

Video: .3gp .mp4

Sensors

Sensors



Motion



Environmental

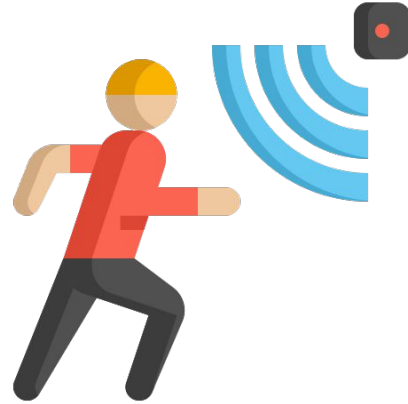


Position

Motion sensors

Measure acceleration forces and rotational forces along three axes:

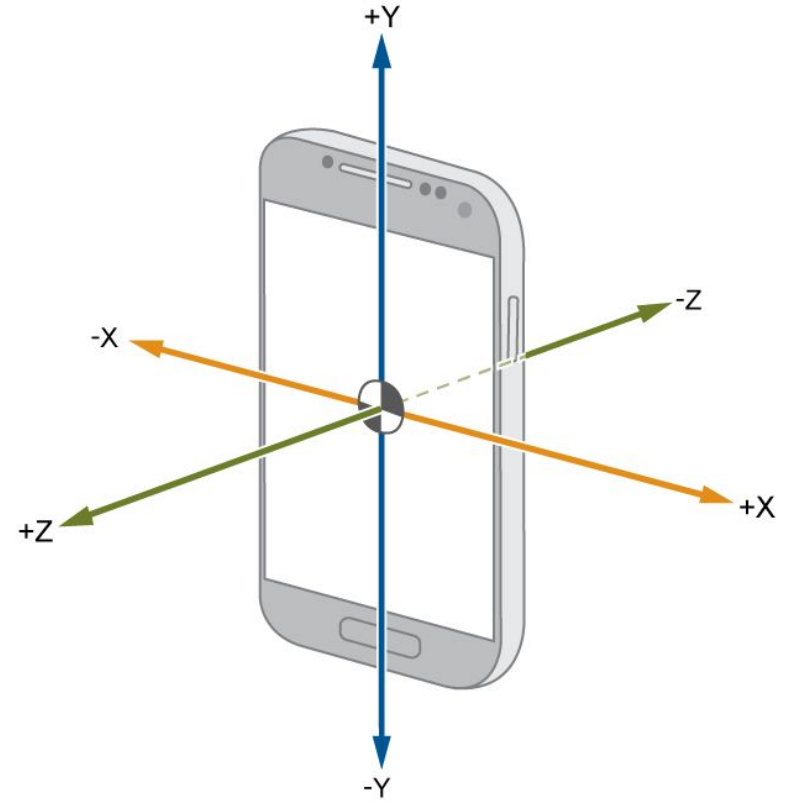
1. Accelerometers
2. Gravity sensors
3. Gyroscopes
4. Rotation vector sensors



Motion sensors

1. Accelerometers

Provides data on the rate of change in **velocity** - the speed in combination with the **direction** of **motion** of an object, excluding gravity

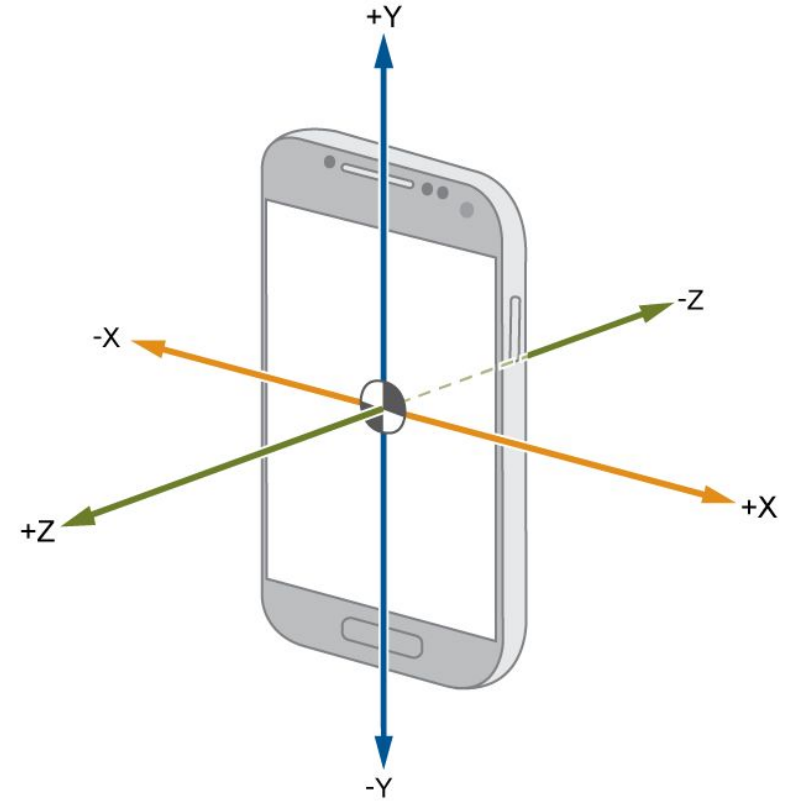


Motion sensors

2. Gravity sensors

It measures the **direction** and **intensity** of gravity

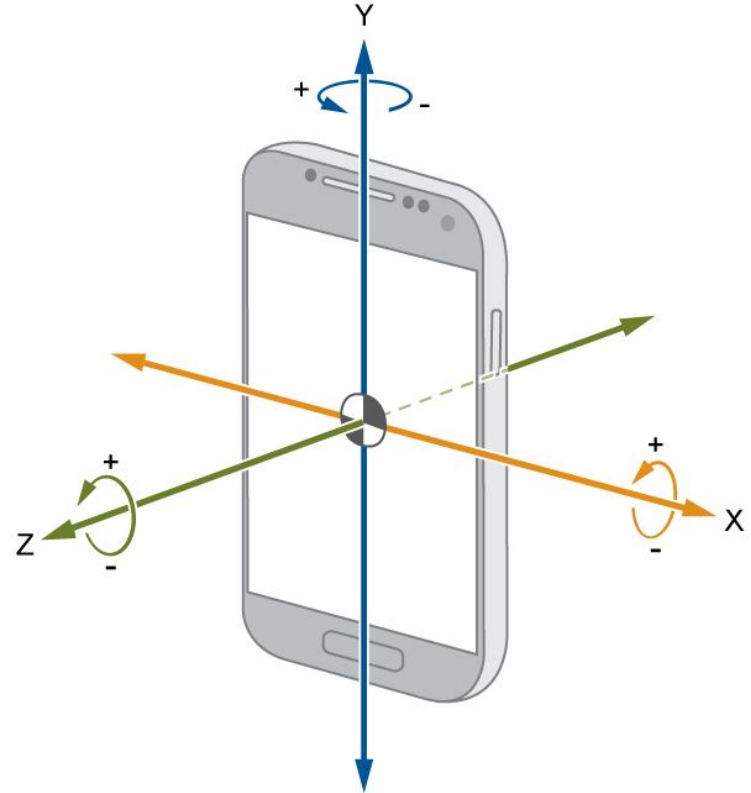
Checks the **relative direction** of a device within a space



Motion sensors

3. Gyroscopes

It helps the accelerometer out with understanding which way your phone is **orientated**.

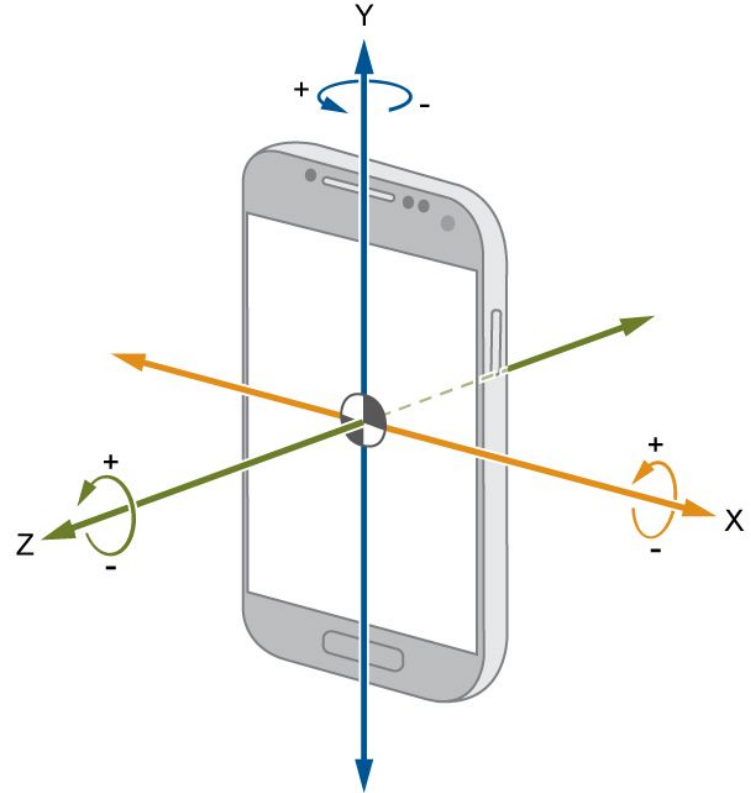


Motion sensors

4. Rotation vector sensors

To monitor and measure **turning movements**

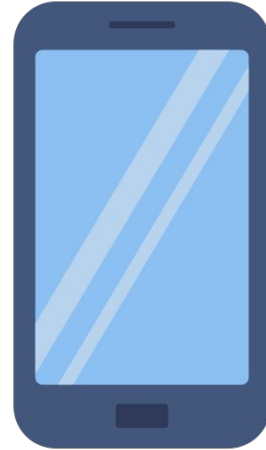
Combination of an angle and an axis



Environmental sensors

Measure various environmental parameters, such as ambient air temperature and pressure, illumination, and humidity **near a mobile device**.

1. Barometers
2. Photometers, and
3. Thermometers

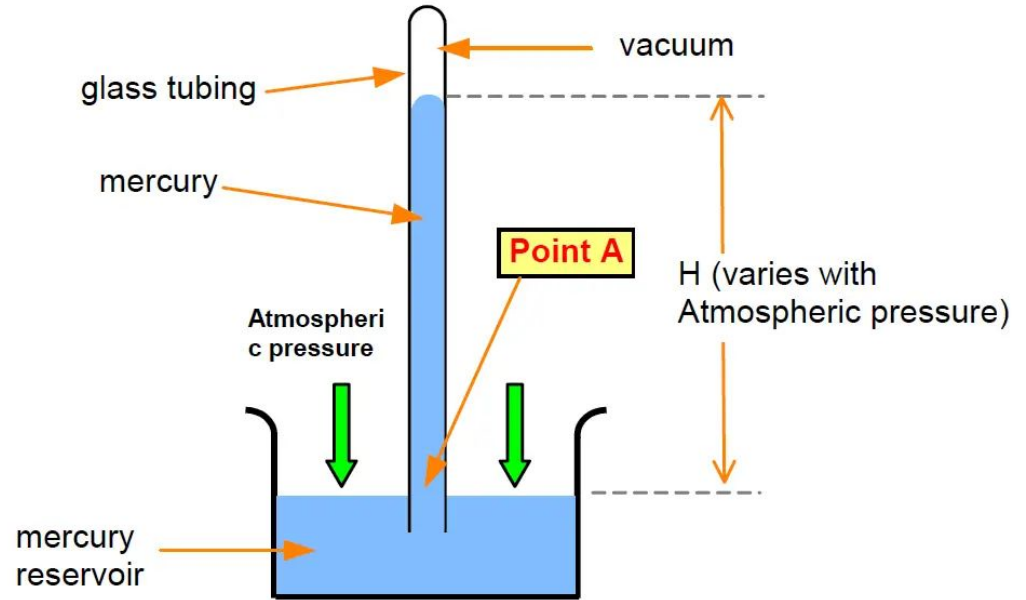


Environmental sensors

Barometer

It measures **atmospheric pressure**

It **assists the GPS** chip inside the device to get a faster lock by instantly delivering **altitude** data (vertical location).

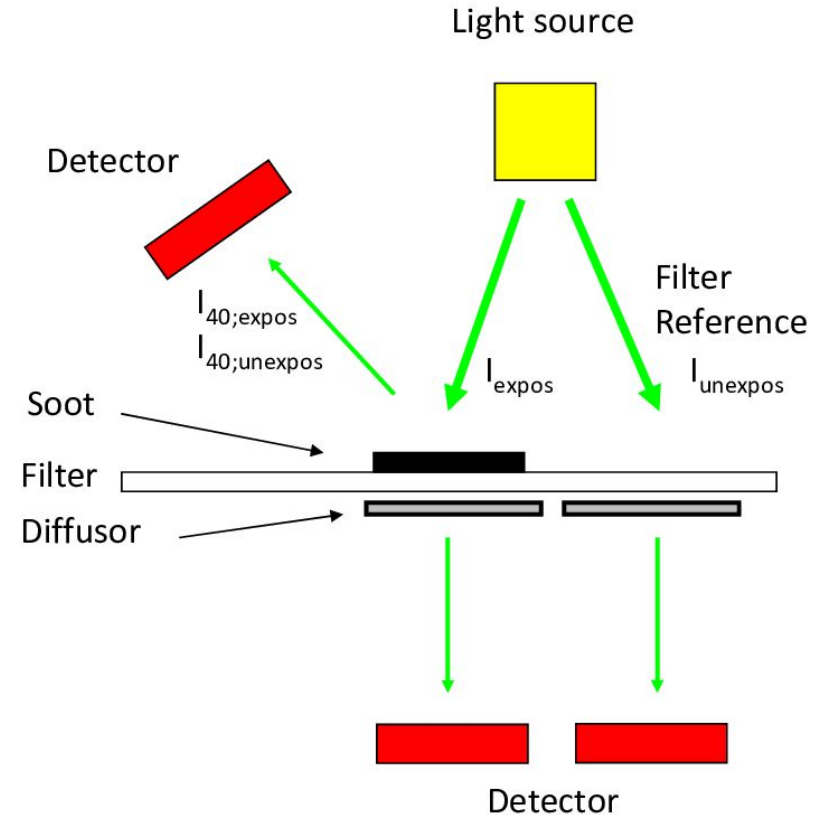


InstrumentationTools.com

Environmental sensors

Photometer / Light sensor

It senses the amount of **ambient light** present

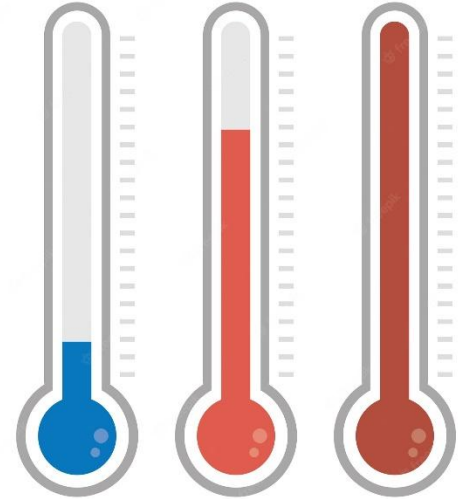


Environmental sensors

Thermometer

It measures **temperature** within a mobile device

To avoid overheating of the internal components



Position sensors

Measure the physical position of a device

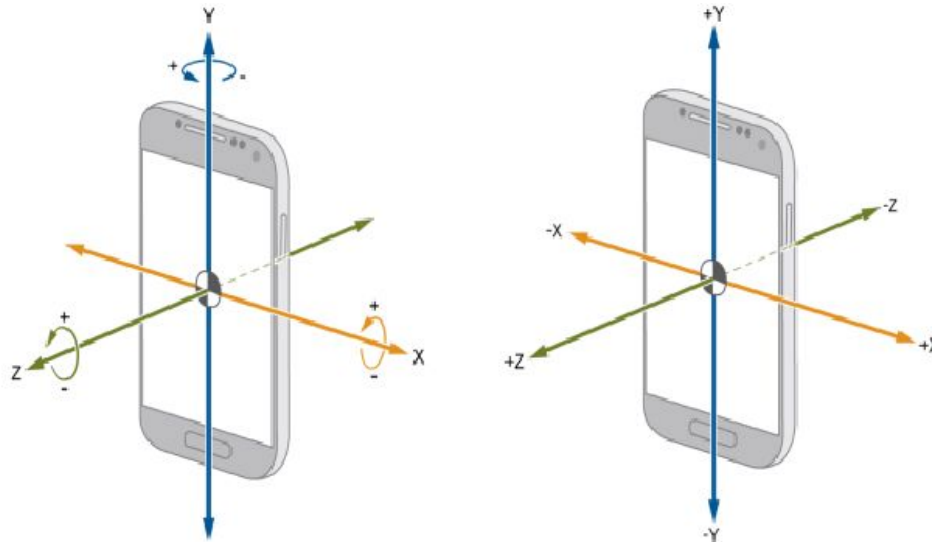
- Orientation sensors
- Magnetometers



Position sensors

Orientation sensors

Measures the orientation of a device relative to an orthogonal coordinate frame



Position sensors

Magnetometers

Determines your location with respect to Magnetic North (or South!)



Other Sensors

Other sensors

Biometrics

Biometric authentication is an approach to multi-factor authentication (MFA) to verify an individual's identity that uses possession of a mobile device as a first factor and use of that device to verify a unique biometric identifier as a second factor.

Flutter has `local_auth` package, which provides local authentication with biometrics such as fingerprint or facial recognition. (Note: supports Android, iOS, macOS and Windows)

Other sensors

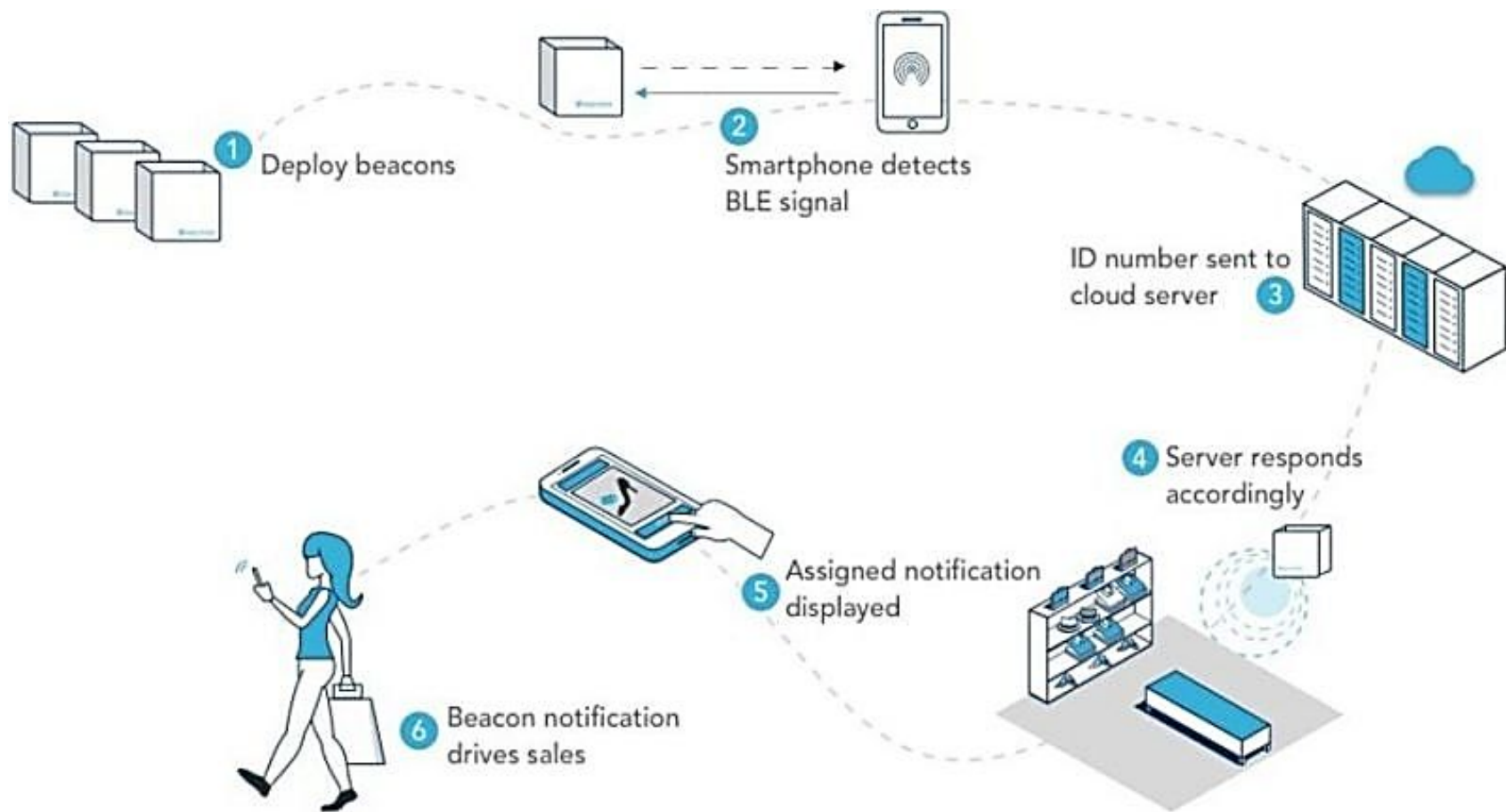
Bluetooth Low Energy (BLE)

A short-range wireless communication technology designed for low-power applications

Low power consumption, high data rate

Applications: wearables, IoT, beacon technology, healthcare, indoor navigation, etc





Other sensors

Near-field communication (NFC)

A short-range wireless technology

Allows two electronic devices to communicate with each other

Applications: mobile payments, data exchange, access control, product information, etc



Near-field communication (NFC)



Other sensors

Nearlink

A short-range wireless communication technology developed by the NearLink Alliance, led by Huawei

High speed, high reliability, high security, low latency, and low energy

Applications: wearable, IoT, automotive, consumer products , etc



NearLink



Technology	BLE	NFC	Nearlink
Range	Up to 100 meters	Up to 4 cm	Up to 100 meters
Speed	Up to 2 Mbps	Up to 424 Kbps	Up to 900 Mbps
Power Consumption	Low	Very Low	Lowest (60% of BLE)
Latency	Moderate	Very Low	Ultra-low

References

Image Picker, https://pub.dev/packages/image_picker

Build your own camera function,
<https://docs.flutter.dev/cookbook/plugins/picture-using-camera>

Audio File, https://pub.dev/packages/just_audio

Video Player, https://pub.dev/packages/video_player

Google ML Kit, https://pub.dev/packages/google_ml_kit

Local authentication, https://pub.dev/packages/local_auth